2026-9-18 Linux网络--TCP拥塞控制与传输优化

- 1. TCP 拥塞控制与传输优化
- 2. 为什么需要拥塞控制
  - 2.1 接收窗口与网络窗口
    - 2.1.1 两个能力限制
    - 2.1.2 共享网络
  - 2.2 拥塞窗口
  - 2.3 从发送窗口到实际发送
- 3. 拥塞控制算法
  - 3.1 慢启动
    - 3.1.1 从小窗口开始探测
    - 3.1.2 慢启动阈值
  - 3.2 拥塞避免
    - 3.2.1 加法增大
    - 3.2.2 网络状态是变化的
  - 3.3 丢包与拥塞信号
    - 3.3.1 超时和重复 ACK
    - 3.3.2 ECN
    - 3.3.3 课堂经验值的边界
- 4. 发送长度与吞吐量
  - 4.1 为什么 4000 字节窗口要拆成多个报文
  - 4.2 带宽时延积
  - 4.3 吞吐量的共同限制
- 5. 延迟应答
  - 5.1 基本原理
  - 5.2 数量限制与时间限制
  - 5.3 延迟应答与窗口
- 6. 捎带应答
  - 6.1 数据与 ACK 合并
  - 6.2 为什么需要两个序号字段
  - 6.3 第三次握手中的捎带
- 7. 优化边界
- 8. 小结

1. TCP 拥塞控制与传输优化

接收窗口只能说明接收端还有多少空间，不能说明网络是否拥堵。TCP 还需估计共享网络当前能承受多少在途数据，这项职责由拥塞控制完成。

本篇关注“发送速度和网络效率”：拥塞窗口、慢启动、拥塞避免、重复 ACK 与网络恢复，以及延迟应答、捎带应答、MSS/MTU 和吞吐量之间的权衡。

前置知识：

- 报文与字段见 2026-9-18 Linux网络--TCP协议基础与报文；
- ACK、序号、重传和接收窗口见 2026-9-18 Linux网络--TCP可靠传输与流量控制；
- 握手交换窗口和连接状态见 2026-9-18 Linux网络--TCP连接管理与异常诊断。

2. 为什么需要拥塞控制

2.1 接收窗口与网络窗口

2.1.1 两个能力限制

发送端同时受到两类限制：

- 接收窗口：对端应用还能接收多少；
- 拥塞窗口：当前网络可以承受多少在途数据。

```
实际发送上限 = min(接收端通告窗口, 拥塞窗口, 当前待发送数据量)
```

接收窗口大，不代表网络一定通畅；网络容量大，也不代表接收端缓冲区还有空间。

2.1.2 共享网络

网络带宽和路由器队列被大量主机共享。出现大面积丢包时，如果所有发送方都立即高速重传，网络负载会进一步增加，形成“丢包 → 重传 → 更拥堵 → 更多丢包”的恶性循环。

拥塞控制让发送方主动降低速度，先让网络恢复，再逐步探测更高吞吐量。

2.2 拥塞窗口

拥塞窗口 `cwnd` 是发送方对网络承载能力的动态估计，单位通常按字节或 MSS 描述。

它不是报头字段，也不会直接发给对端。它是发送端内核根据 ACK、丢包、重复 ACK、ECN 等信息维护的内部状态。

2.3 从发送窗口到实际发送

发送缓冲区中的滑动窗口、接收窗口和拥塞窗口之间的关系可以简化为：

```
发送范围受滑动窗口组织
接收窗口限制对端剩余空间
拥塞窗口限制网络中允许在途的数据量
最终发送量取三者可用范围的较小值
```

滑动窗口如何维护未确认数据，见 2026-9-18 Linux网络--TCP可靠传输与流量控制。

3. 拥塞控制算法

3.1 慢启动

3.1.1 从小窗口开始探测

连接刚开始或经历严重超时后，发送方不能立即按最大窗口发送，而应先用较小窗口探测网络。

每收到有效的累计确认，拥塞窗口逐渐增加。经典慢启动表现为每个 RTT 窗口近似翻倍，因此曲线在前期看起来增长较慢，进入增长阶段后很快上升。

3.1.2 慢启动阈值

慢启动阈值 `ssthresh` 表示从“快速增长”切换到“谨慎增长”的参考边界。

```
cwnd < ssthresh：慢启动，窗口近似指数增长
cwnd >= ssthresh：拥塞避免，窗口近似线性增长
```

`ssthresh` 和 `cwnd` 都会随网络状态改变，不存在适用于所有网络的固定最优值。

3.2 拥塞避免

3.2.1 加法增大

达到 `ssthresh` 后，发送方不再让窗口持续快速翻倍，而是逐步增加，避免很快再次越过网络容量。

经典模型用“加法增大”描述稳定阶段，用“乘法减小”描述发生拥塞后的退让。

3.2.2 网络状态是变化的

白天、夜间、不同区域和不同链路的负载都不同，因此当前可承受窗口持续变化。TCP 只能不断尝试：

- 顺利时增加发送能力；
- 出现拥塞信号时降低窗口；
- 网络恢复后重新探测。

拥塞控制的本质是探索网络当前的安全发送能力，而不是找到一个永久不变的数值。

3.3 丢包与拥塞信号

3.3.1 超时和重复 ACK

超时通常意味着长时间没有收到新的确认，常被当作较严重的拥塞信号。发送方会明显降低窗口并重新探测。

重复 ACK 可能表示某个数据段缺失，也可能反映网络已经发生局部丢包。不同 TCP 算法会采用快速重传、快速恢复或减小窗口等策略。

3.3.2 ECN

显式拥塞通知 ECN 允许网络设备在拥塞时标记报文，而不必直接丢弃。接收端通过 ECE 将拥塞信息反馈给发送端，发送端再用 CWR 表示已经响应。

ECN 是拥塞通知机制，不代表所有网络设备和路径都启用。

3.3.3 课堂经验值的边界

“少量丢包只重传，大量丢包视为拥塞”和“3% 到 5% 丢包率非常拥堵”适合课堂理解，但不是所有材质、链路和 TCP 实现的通用阈值。

无线误码、硬件故障、策略丢包也可能产生丢失。判断原因应结合抓包、重传率、RTT、ECN 和应用吞吐量。

4. 发送长度与吞吐量

4.1 为什么 4000 字节窗口要拆成多个报文

窗口按字节描述总发送范围，MSS 描述一个 TCP 报文最多可携带多少有效载荷，MTU 又限制链路层帧的总大小。

```
窗口 = 4000 字节
MSS  = 1460 字节（以太网常见示例）
实际 = 拆成多个 TCP 段
```

所以窗口大不等于单个 TCP 数据段可以无限大。IP 分片也不应被当作扩大 TCP 段的常规手段。

4.2 带宽时延积

高带宽、长时延链路需要足够的在途数据才能填满链路。窗口小于链路容量时，吞吐量主要受窗口限制；窗口足够大时，才可能受带宽和协议开销限制。
拥塞窗口和接收窗口都需要足够大，单方面扩大一个窗口并不能必然提升吞吐量。

4.3 吞吐量的共同限制

发送效率至少受以下因素影响：

- 对端接收窗口；
- 网络拥塞窗口；
- MSS/MTU；
- 往返时延 RTT；
- 丢包与重传；
- 应用读写速度和 socket 缓冲区；
- 延迟 ACK 和应用层处理模式。

TCP 优化不能只看一个参数。

5. 延迟应答

5.1 基本原理

接收方收到数据后，不一定立即发送 ACK。适当等待可以：

- 把多个 ACK 合并；
- 等待应用读走数据；
- 在下一份 ACK 中通告更大的接收窗口。

累计确认允许后一个 ACK 覆盖前面连续数据的确认，因此延迟 ACK 不必逐段回复。

5.2 数量限制与时间限制

常见策略为：

```
收到 N 个数据段后必须确认
或
等待时间达到上限后必须确认
```

具体 N 和等待时间由操作系统实现决定。课堂中的“2 个报文”或“200 ms”是典型参数，不是 TCP 规范固定值。

出现乱序、缺口或控制状态变化时，应及时发送 ACK，不能为了合并而延迟丢包恢复。

5.3 延迟应答与窗口

接收端等待期间，应用可能把数据读走，接收缓冲区剩余空间变大。等下一份 ACK 发出时，窗口字段可以通告更大的值。

不过，接收窗口变大不代表实际吞吐一定上升；如果拥塞窗口很小，发送仍会受网络侧限制。

6. 捎带应答

6.1 数据与 ACK 合并

TCP 是全双工的。双方同时有数据要发送时，可以把确认和本端数据放进同一个 TCP 报文：

```
seq     = 本端发送数据的起始序号
ack_seq = 对端数据的下一个期望序号
ACK     = 1
data    = 本端有效载荷
```

接收方看到的报文既确认了自己的数据，又带来了新的业务数据。

6.2 为什么需要两个序号字段

`seq` 表示本端发送进度，`ack_seq` 表示本端接收进度。如果只保留一个序号字段，就无法同时描述双向通信的两个独立进度。

因此，TCP 可以同时进行“发送自己的数据”和“确认对端的数据”。

6.3 第三次握手中的捎带

TCP 在第三次握手 ACK 到达后即可进入 ESTABLISHED。某些实现允许第三次 ACK 携带应用数据，以减少一次往返。
但是否携带数据还受到实现、协议扩展和服务端接收策略影响，不能把 SYN 或第三次 ACK 一律当作普通业务数据通道。

7. 优化边界

TCP 的性能优化不是把所有窗口无限放大，也不是见到丢包就高频重传。合理目标是：

```
网络不拥塞时充分利用在途数据
出现拥塞时及时降低发送速度
丢包后尽快恢复
避免无意义 ACK 和重传
保持连接状态可控
```

不同 Linux 内核版本、拥塞控制算法和网络设备会采用不同参数。课堂经验值适合建立直觉，实际生产系统应结合 `ss -ti`、`tcpdump`、应用延迟和吞吐量观测。

8. 小结

```
共享网络
  -> 大量并发发送需要节制
  -> 拥塞窗口动态探测网络承载力
  -> 慢启动快速找到大致区间
  -> 拥塞避免逐步增长
  -> 丢包/ECN 信号触发降速和恢复

应用侧优化
  -> 延迟应答合并确认并扩大窗口
  -> 捎带应答减少纯 ACK
  -> MSS/MTU 决定单个报文承载能力
```

拥塞控制关注网络，流量控制关注接收端；滑动窗口负责组织未确认数据，延迟和捎带应答提高报文利用率。只有把这些边界分清，才能避免把性能问题错误归因于某一个窗口或某一个ACK。